

Competitors, Customization & Data Migration

Engineering Review · 16 Aug 2026

August 16, 2026

CONTENTS

Competitor Weaknesses, User Customization, and Data Migration

1. Competitor weaknesses

1.1 The landscape that actually matters

1.2 Weakness map

1.3 The five weaknesses worth building against

1.4 Where competitors are strong — do not fight here

2. User customization

2.1 The customization feature set

2.2 Schema for customization

2.3 Resolution and safety rules

3. Data migration

3.1 Sources and their realities

3.2 What actually gets migrated

3.3 Pipeline

3.4 The failure modes that actually bite

3.5 Migration tables

3.6 Core tables the migration lands in

3.7 Migration as a product, not a service

4. What to do with this

Competitor Weaknesses, User Customization, and Data Migration

Companion to [REVIEW.md](#), [ROADMAP.md](#), and [ODOO-COMPARISON.md](#).

Three questions, answered in order:

1. **Where are the competitors weak** — and which of those weaknesses is a wedge rather than a rounding error.
2. **What customization users need** — the concrete feature set, and a schema that delivers it without per-tenant DDL.
3. **Data migration** — the pipeline, the failure modes, and the DB tables for both the migration machinery and the core ledger it lands in.

Product facts about competitors move fast (editions, plan limits, India module coverage). Treat Section 1 as a research brief to verify before it goes on a pricing page, not as citable fact.

1. Competitor weaknesses

1.1 THE LANDSCAPE THAT ACTUALLY MATTERS

Odoo is not the main opponent. For a cloud multi-branch Indian GST product the real field is:

Product	Shape	Who buys it
Tally Prime	Desktop, per-machine licence, the incumbent	Almost every Indian SMB, and their CA
Busy	Desktop, distribution-heavy, multi-location	Distributors, wholesalers
Marg ERP	Desktop, vertical (pharma, FMCG)	Pharma distribution
Zoho Books	Cloud, polished, cheap, India-native	Modern SMBs, services, D2C
Vyapar / myBillBook	Mobile-first billing	Micro-businesses, retail counters
Odoo	Full ERP	Mid-market with a consultant budget
ClearTax / GSP filing tools	Compliance only	Anyone filing GST

1.2 WEAKNESS MAP

Weakness	Tally	Busy	Marg	Zoho Books	Vyapar	Odoo
True cloud, multi-user, anywhere access	Weak (hosted RDP via partners)	Weak	Weak	Strong	Medium	Strong
Branch/warehouse-level <i>access control</i>	Weak	Medium	Medium	Weak	None	Weak
Multi-branch consolidation without a sync product	Weak	Medium	Medium	Medium	None	Medium
Granular role permissions	Coarse	Coarse	Coarse	Medium	None	Strong
Modern UI / low training cost	Weak (but fast for trained operators)	Weak	Weak	Strong	Strong	Medium
Mobile	Weak	Weak	Weak	Strong	Strong	Medium
Open API / integrations	Weak (TDL/XML)	Weak	Weak	Strong	Weak	Strong
Real accounting depth	Strong	Strong	Strong	Strong	Weak	Strong
Audit trail / edit log quality	Medium	Medium	Medium	Medium	Weak	Strong
Per-user pricing pain	n/a (per machine)	n/a	n/a	High	Low	High
Getting your data <i>out</i>	Weak	Weak	Weak	Medium	Weak	Medium

Weakness	Tally	Busy	Marg	Zoho Books	Vyapar	Odoo
Accountant multi-client console	Weak	Weak	Weak	Medium	Weak	Medium
Poor-connectivity operation	Strong (local)	Strong	Strong	Weak	Medium	Weak

1.3 THE FIVE WEAKNESSES WORTH BUILDING AGAINST

Most gaps above are not wedges — they are features a competitor ships next quarter. These five are structural, meaning the incumbent cannot fix them without breaking their own model:

1. Branch and warehouse as an access boundary. Everyone models branches as either separate books (Tally companies, Odoo companies) or as a reporting tag (analytic accounts, cost centres). Nobody makes “this storekeeper sees only Warehouse 3 of Branch 2, and every voucher they touch is scoped to it” a first-class, enforced concept. **This app already does — it is the single strongest asset in the codebase.** Lead with it.

2. Leaving Tally is deliberately hard. No competitor invests in making migration painless, because they all assume you arrive from a spreadsheet. Migration quality is a *product*, not a services task — see Section 3. Whoever makes “switch from Tally in one day, balances reconciled to the paisa” true owns the category’s biggest friction point.

3. Per-user pricing punishes exactly your buyer. A 4-branch distributor with 16 people who each touch a screen is priced out of Zoho and Odoo. Flat per-branch pricing with unlimited users is a message the incumbents cannot match without repricing their whole book.

4. Desktop incumbents are weak on connectivity-tolerant *cloud*. Tally works offline because it is local; Zoho fails on a bad line because it is not. A cloud product that keeps working through a dropped connection and syncs afterward beats both. This app has accidental offline behaviour today (localStorage) — done deliberately, with a sync queue and conflict rules, it becomes a headline feature rather than a data-loss bug.

5. The CA is the channel, and nobody serves them well. Chartered accountants recommend the software, do the filings, and manage 30–200 client books. A real multi-client console — one login, client switcher, per-client permissions, bulk filing status, “which clients haven’t reconciled yet” — is the highest- leverage feature in this market and it is nobody’s flagship.

1.4 WHERE COMPETITORS ARE STRONG — DO NOT FIGHT HERE

- **Tally’s data-entry speed and CA familiarity.** Every accountant in India knows the keyboard flow. Match the muscle memory instead of arguing with it: same shortcuts, same voucher vocabulary (F5 payment, F6 receipt, F8 sales).
 - **Zoho’s polish, ecosystem, and support.** Do not out-feature Zoho on generic invoicing; out-specialise them on multi-branch inventory + branch access.
 - **Vyapar’s simplicity for a one-person shop.** That customer is not yours.
 - **Odoo’s breadth.** Concede manufacturing, HR, ecommerce, projects.
-

2. User customization

Customization is what turns a demo into a system a business will not leave. Two rules before the feature list:

- **No per-tenant DDL.** Never create tables or columns per customer. Definitions in metadata tables, values in JSON columns, validated at the application layer.
- **Everything customizable is scoped.** Every customization row carries `accountId` + `orgId` and optionally `branchId`, resolving `branch` → `org` → `default`. This mirrors the tenancy model already enforced in `requireTenantContext`.

2.1 THE CUSTOMIZATION FEATURE SET

#	Function	What the user does	Priority
C1	Custom fields	Add fields to customers, vendors, items, invoices, bills, etc. Types: text, number, date, dropdown, checkbox, party/item reference. Mark required, searchable, print-on-document	P0
C2	Document numbering series	Per document type, per branch, per financial year: prefix, suffix, padding, reset rule, manual override policy	P0 (partially exists, client-side)
C3	Print/PDF template designer	Pick layout, logo, colour, which columns show, terms text, signature block, bank details, per doc type per branch. Multiple templates, one default	P0 (basic version exists)
C4	Custom masters	User-defined lists (item category, customer group, salesperson, route, transporter) used as dropdowns and report filters	P1
C5	Role designer + record rules	Beyond module/action: restrict by branch, warehouse, customer group, document value ceiling; field-level read/write	P1 (module/action layer exists)
C6	Approval workflows	Rule builder: <i>when</i> invoice discount > 10% <i>or</i> value > ₹5,00,000 → require approval by role X, block posting until approved	P1
C7	Saved views	Per list: chosen columns, filters, sort, grouping; save, name, share with a role, set as default	P1
C8	Dashboard builder	Choose widgets, arrange, scope to branch, set as landing page per role	P2

#	Function	What the user does	Priority
C9	Message templates	Email/WhatsApp/SMS bodies with merge fields (<code>{{invoice.number}}</code> , <code>{{party.name}}</code>), per document type, per language	P1
C10	Automation rules	Trigger (document posted, payment overdue by N days, stock below reorder level) → action (send template, create task, notify role, webhook)	P2
C11	Terms, payment terms, price lists	Named payment terms (Net 30, 50% advance), per-customer price lists, discount policies	P1
C12	Form layouts	Per role: hide fields, reorder, mark mandatory, set defaults	P2
C13	Import mappings	Save a column mapping from a customer's Excel layout and reuse it every month	P1 (also used by migration, Section 3)
C14	API keys + webhooks	Customer's own integrations, scoped tokens, event subscriptions	P2
C15	Regional settings	Fiscal year start, date format, number format (Indian lakh/crore grouping), rounding policy, decimal places, language	P0
C16	Voucher type configuration	Clone a document type with its own numbering, template, approval, and default accounts (Tally users expect this)	P2

2.2 SCHEMA FOR CUSTOMIZATION

Add to `server/prisma/schema.prisma`. Every model is org-scoped; `branchId` null means “applies org-wide”.

```
// ----- C1: custom fields -----
model CustomFieldDef {
  id          String  @id @default(cuid())
  accountId   String
  orgId       String
  branchId    String?      // null = all branches

  entity      String      // "Customer" | "Item" | "Invoice" | ...
  key         String      // stable machine name, e.g. "route_code"
  label       String
  fieldType   String      // TEXT|NUMBER|DATE|SELECT|BOOLEAN|REF
  options     String?     // JSON array for SELECT
  refEntity   String?     // for REF: "Customer" | "Item"
  required    Boolean @default(false)
  searchable  Boolean @default(false)
  printable   Boolean @default(false)
  defaultValue String?
  sortOrder   Int        @default(0)
  isActive    Boolean @default(true)

  createdByUserId String
  createdAt     DateTime @default(now())
  updatedAt     DateTime @updatedAt

  @@unique([orgId, entity, key])
  @@index([accountId, orgId, entity])
}

// Values live on the record itself in a JSON column named customFields.
// On Postgres make it Json and add a GIN index for the searchable keys:
//   model Invoice { ... customFields Json? }
//   CREATE INDEX invoice_custom_fields_gin ON "Invoice" USING gin
// ("customFields");
// Validation against CustomFieldDef happens in the service layer, never
// in SQL.

// ----- C2: numbering -----
model NumberSeries {
  id          String  @id @default(cuid())
  accountId   String
  orgId       String
  branchId    String?
  docType     String      //
  INVOICE|BILL|RECEIPT|PAYMENT|JOURNAL|TRANSFER...
  name        String
```

```

    prefix      String @default("")
    suffix      String @default("")
    padding     Int     @default(4)
    nextNumber  Int     @default(1)
    resetPolicy String @default("FISCAL_YEAR") //
NEVER|FISCAL_YEAR|MONTH
    fiscalYear  String? // "2026-27" when reset
by FY
    allowManual Boolean @default(false)
    isDefault   Boolean @default(false)
    isActive    Boolean @default(true)

    @@unique([orgId, branchId, docType, name, fiscalYear])
    @@index([accountId, orgId, docType])
}
// Allocation must be atomic and gap-free:
//  UPDATE "NumberSeries" SET "nextNumber" = "nextNumber" + 1
//  WHERE id = $1 RETURNING "nextNumber" - 1;
// inside the same transaction that inserts the document. Never mint
numbers
// client-side (today's bumpCompanyNextNumber does exactly that).

// ----- C3: print templates -----
model DocTemplate {
    id          String @id @default(cuid())
    accountId   String
    orgId       String
    branchId    String?
    docType     String
    name        String
    layout      String // "classic" | "modern" | "compact" | ...
    config      String // JSON: logoUrl, accent,
visibleColumns[], // termsText, showBankDetails,
signatureLabel...
    isDefault   Boolean @default(false)
    isActive    Boolean @default(true)

    @@unique([orgId, branchId, docType, name])
}

// ----- C4: custom masters -----
model CustomListDef {
    id          String @id @default(cuid())
    accountId   String
    orgId       String

```

```

key      String          // "item_category" | "sales_route"
label    String
appliesTo String          // entity this list decorates
values   CustomListValue[]

@@unique([orgId, key])
}

model CustomListValue {
  id      String @id @default(cuid())
  accountId String
  orgId   String
  listId  String
  code    String
  label   String
  sortOrder Int @default(0)
  isActive Boolean @default(true)

  list CustomListDef @relation(fields: [listId], references: [id],
onDelete: Cascade)

  @@unique([listId, code])
}

// ----- C5: record-level rules -----
model AccessRule {
  id      String @id @default(cuid())
  accountId String
  orgId   String
  roleId  String
  entity  String
  // JSON predicate evaluated server-side and compiled into the Prisma
  where
  // clause, e.g. {"warehouseId":{"in":"$user.allowedWarehouseIds"}}
  predicate String
  canRead   Boolean @default(true)
  canWrite  Boolean @default(false)
  maxValue  Decimal? // document value ceiling

  @@index([accountId, orgId, roleId, entity])
}

// ----- C6: approvals -----
model ApprovalRule {
  id      String @id @default(cuid())
  accountId String

```

```

    orgId      String
    branchId   String?
    docType    String
    name       String
    condition   String          // JSON: {"total":{"gt":500000}}
    approverRoleId String
    sequence    Int      @default(1) // multi-step chains
    blocksPosting Boolean @default(true)
    isActive    Boolean @default(true)

    @@index([accountId, orgId, docType])
}

model ApprovalRequest {
    id          String @id @default(cuid())
    accountId   String
    orgId       String
    branchId    String
    docType     String
    docId       String
    ruleId      String
    sequence    Int
    status      String @default("PENDING") // PENDING|APPROVED|REJECTED
    decidedByUserId String?
    decidedAt   DateTime?
    comment     String?
    createdAt   DateTime @default(now())

    @@index([accountId, orgId, docType, docId])
    @@index([accountId, orgId, status])
}

// ----- C7/C8/C12: saved views, dashboards, form layouts -----
model UiPreset {
    id          String @id @default(cuid())
    accountId   String
    orgId       String
    kind        String          // SAVED_VIEW | DASHBOARD | FORM_LAYOUT
    scopeKey    String          // list/screen identifier, e.g.
    "invoices.list"
    name        String
    config      String          // JSON
    ownerUserId String?         // null = shared
    roleId      String?         // shared with a role
    isDefault   Boolean @default(false)

```

```

    @@index([accountId, orgId, kind, scopeKey])
}

// ----- C9/C10: templates and automations -----
model MessageTemplate {
  id          String @id @default(cuid())
  accountId   String
  orgId       String
  channel     String           // EMAIL | WHATSAPP | SMS
  docType     String
  name        String
  subject     String?
  body        String           // merge fields: {{invoice.number}}
  language    String @default("en")
  isDefault   Boolean @default(false)

  @@unique([orgId, channel, docType, name, language])
}

model AutomationRule {
  id          String @id @default(cuid())
  accountId   String
  orgId       String
  branchId    String?
  trigger     String           // DOC_POSTED | PAYMENT_OVERDUE |
STOCK_BELOW_MIN
  condition   String?         // JSON
  action      String           // SEND_TEMPLATE | WEBHOOK | NOTIFY_ROLE
  actionConfig String         // JSON
  isActive    Boolean @default(true)

  @@index([accountId, orgId, trigger])
}

// ----- C15: regional settings -----
model OrgSetting {
  id          String @id @default(cuid())
  accountId   String
  orgId       String
  branchId    String?
  key         String           // "fiscal_year_start" | "number_format" |
...
  value       String           // JSON scalar or object

```

```

    @@unique([orgId, branchId, key])
  }

```

2.3 RESOLUTION AND SAFETY RULES

- **Resolution order** for any customization: branch-specific row → org row → system default. Cache per (org, branch) with a version counter bumped on write; never query these tables per record render.
- **Custom field keys are immutable** once data exists. Renaming the *label* is free; renaming the *key* is a migration.
- **Deleting a definition soft-deletes** (`isActive = false`). Historical documents keep their values and still print them.
- **Predicates and conditions are JSON, never expressions.** Do not build an eval-based rule engine — it becomes a remote code execution hole. Compile a fixed operator set (`eq, ne, gt, gte, lt, lte, in, contains`) into Prisma `where` clauses server-side.
- **Everything customizable is exportable.** A tenant's customization set should round-trip as JSON so support can reproduce a customer's configuration.

3. Data migration

This is the correct thing to be worried about. Migration is where switching customers are won and where a bad first week loses them permanently. Two principles set everything else:

1. **The cutover is a balance sheet, not a history dump.** Bring masters, open items, and opening balances. Historical detail is optional and usually only the current financial year.
2. **Every migration is a dry run until the numbers match.** Nothing writes to live tables until a reconciliation report proves source and target agree.

3.1 SOURCES AND THEIR REALITIES

Source	Extraction path	Practical notes
Tally Prime	XML export (Masters / Day Book / Trial Balance), or HTTP XML request to the running Tally on port 9000, or ODBC	The XML is verbose and Tally-idiomatic. Ledger <i>groups</i> carry the accounting meaning. Expect Unicode name variants and duplicate ledgers
Busy	Excel/CSV export per master and register; some XML	Multi-location data needs per-location extraction
Marg	Excel/CSV export, DB access on-premise	Vertical fields (batch, expiry) matter for pharma
Zoho Books	CSV export per module, or REST API	Cleanest source; has stable IDs to map against
QuickBooks (India exits)	CSV/QBO export	Legacy migrations still show up
Vyapar / myBillBook	Excel export	Shallow data; mostly parties + items + open invoices
Excel / manual	Your own templates	The most common path — make the templates excellent

Build **generic Excel/CSV ingestion first** (it covers every source through a manual export), then a Tally XML parser (the biggest single audience), then a Zoho API connector.

3.2 WHAT ACTUALLY GETS MIGRATED

Wave	Data	Notes
1	Chart of accounts / ledger groups	Needs a mapping table from source group → your account type/group
2	Parties (customers, vendors)	Dedupe on GSTIN, then PAN, then normalized name+phone
3	Items, UoM, HSN, tax rates	UoM conflicts and missing HSN are the top two blockers
4	Warehouses / branches / godowns	Maps to your <code>Branch</code> + <code>Warehouse</code>
5	Opening balances (trial balance as of cutover date)	Posts one balanced opening journal entry
6	Open items: unpaid invoices, unpaid bills, advances	Needed for aged AR/AP to be correct on day one
7	Stock on hand with quantity and value per item per warehouse	Must tie to the Stock-in-Hand opening balance
8	Historical transactions (optional)	Current FY only, unless the customer insists
9	Attachments, price lists, custom fields	Last

3.3 PIPELINE

```
upload → parse → profile → map → validate (dry run) → stage
      → commit (single transaction per wave) → reconcile → report
                                         ↳ rollback (by batch)
```

- **Upload.** File stored with a checksum; never re-parsed from the user's disk.
- **Parse.** Source-specific adapter emits *canonical rows* (one shape per entity, regardless of source).
- **Profile.** Show the user what was found: 412 parties, 1,203 items, 87 ledgers, 14 warehouses, and every column detected. This screen builds trust.

- **Map.** Column → field mapping, plus value mapping (source ledger group → your account group; source UoM → your UoM). Saved as a reusable `MigrationMapping` (shared with customization C13).
- **Validate.** Every row checked without writing: required fields, GSTIN format and checksum, duplicate detection, unknown references, negative stock, dates outside the fiscal year, debits ≠ credits. Output is a per-row issue list the user can fix in place or re-upload.
- **Stage.** Rows land in `MigrationStagingRecord` with a resolved payload.
- **Commit.** Per wave, inside one DB transaction, writing both the real records and an `MigrationEntityMap` row per created entity. Idempotent: re-running a committed batch is a no-op because the external key already maps.
- **Reconcile.** Automated comparison, not eyeballing:
 - trial balance total debits/credits vs source → must be **exactly zero** difference
 - per-party outstanding vs source aged report
 - stock quantity and value per warehouse vs source stock summary
 - GST tax totals for the period, if history was imported
- **Report.** A signed-off PDF the customer's CA can keep. This is a sales asset.
- **Rollback.** Delete by `batchId`, in reverse dependency order, allowed only while the batch is `COMMITTED` and no post-cutover documents reference it.

3.4 THE FAILURE MODES THAT ACTUALLY BITE

Problem	Handling
Duplicate parties (“ABC Traders”, “A.B.C. Traders Pvt Ltd”)	Fuzzy match on normalized name + GSTIN/PAN/phone; present as a merge screen, never auto-merge
Tally group → your account type has no clean mapping	Ship a default mapping table for the standard Tally groups; force explicit user choice for anything unmapped. Do not silently dump to Suspense
Opening trial balance does not balance in the source	Block commit. Show the difference and let the user post a documented adjustment to a named Opening Difference account
Stock value present but no per-item cost	Derive rate = value/qty; flag items where qty = 0 but value ≠ 0
Items with the same name, different UoM	Treat (name, UoM) as the key; surface conflicts for user decision
Missing HSN/GST rate	Import with a flag; block e-Invoice/GSTR export on flagged items until fixed
Invoice numbers colliding with your number series	Import historical numbers as-is with <code>allowManual</code> ; set <code>nextNumber</code> above the highest imported
Rounding drift between source and target	Reconcile at the paisa; carry a per-document rounding difference field
Re-run after a partial failure	Batch + external key mapping makes every wave idempotent
Customer keeps working in Tally during the migration	Support a delta run: same batch source, only rows after a watermark

3.5 MIGRATION TABLES

```

model MigrationBatch {
    id            String    @id @default(cuid())
    accountId     String
    orgId         String
    branchId      String?

    sourceSystem  String    //
    TALLY|BUSY|MARG|ZOH0|QUICKBOOKS|VYAPAR|EXCEL
    sourceVersion String?
    cutoverDate   String    // opening balances as of this date
    fiscalYear    String

    status        String    @default("DRAFT")
    //
    DRAFT|PARSING|MAPPING|VALIDATING|STAGED|COMMITTING|COMMITTED|FAILED|ROLLED_E
    mode          String    @default("DRY_RUN") // DRY_RUN | COMMIT
    waves         String    // JSON: which waves this batch covers

    totalRows     Int       @default(0)
    validRows     Int       @default(0)
    errorRows     Int       @default(0)
    committedRows Int       @default(0)

    startedAt     DateTime?
    committedAt   DateTime?
    rolledBackAt  DateTime?

    createdByUserId String
    createdAt     DateTime @default(now())
    updatedAt     DateTime @updatedAt

    files         MigrationFile[]
    records       MigrationStagingRecord[]
    issues        MigrationIssue[]
    entityMap     MigrationEntityMap[]
    recon         MigrationReconciliation[]

    @@index([accountId, orgId, status])
}

model MigrationFile {
    id            String    @id @default(cuid())
    accountId     String

```

```

orgId      String
batchId     String

filename     String
storageKey   String           // object storage key, not a local path
contentType  String
sizeBytes    Int
checksum     String           // sha256, for idempotency
entityHint   String?          // "parties" | "items" | "trial_balance"
rowCount     Int               @default(0)
parsedAt     DateTime?

    batch MigrationBatch @relation(fields: [batchId], references: [id],
onDelete: Cascade)

    @@unique([batchId, checksum])
    @@index([accountId, orgId, batchId])
}

// Reusable column/value mapping – also powers customization C13.
model MigrationMapping {
    id          String @id @default(cuid())
    accountId    String
    orgId        String

    sourceSystem String
    entity        String           // "Party" | "Item" | "TrialBalance" |
    ...
    name          String
    // JSON: {"columns":{"Party Name":"name","GSTIN":"gstn"},
    //       "values":{"group":{"Sundry Debtors":"<accountGroupId>"}}}
    config        String
    isDefault      Boolean @default(false)

    createdByUserId String
    createdAt      DateTime @default(now())
    updatedAt      DateTime @updatedAt

    @@unique([orgId, sourceSystem, entity, name])
}

// One row per incoming record, before it becomes a real entity.
model MigrationStagingRecord {
    id          String @id @default(cuid())
    accountId    String
    orgId        String

```

```

    batchId      String

    entity        String          //
Party|Item|Account|OpeningBalance|OpenInvoice|Stock
    rowNumber     Int
    sourceKey      String?          // source system's own id/GUID, when it
has one
    rawPayload     String          // JSON as parsed
    mappedPayload  String?         // JSON after mapping, pre-validation
    status         String          @default("PENDING")
// PENDING|VALID|INVALID|SKIPPED|COMMITTED|FAILED
    targetId      String?         // filled on commit
    errorText      String?

    batch MigrationBatch @relation(fields: [batchId], references: [id],
onDelete: Cascade)

    @@index([accountId, orgId, batchId, entity, status])
    @@index([batchId, entity, sourceKey])
}

// The idempotency backbone: source identity → your identity.
model MigrationEntityMap {
    id            String @id @default(cuid())
    accountId     String
    orgId         String
    batchId       String

    sourceSystem  String
    entity        String
    sourceKey      String          // Tally GUID, Zoho id, or a normalized
natural key
    targetId      String

    createdAt     DateTime @default(now())

    batch MigrationBatch @relation(fields: [batchId], references: [id])

    @@unique([orgId, sourceSystem, entity, sourceKey])
    @@index([accountId, orgId, entity, targetId])
}

model MigrationIssue {
    id            String @id @default(cuid())
    accountId     String
    orgId         String

```

```

batchId    String
recordId   String?

severity    String          // ERROR | WARNING | INFO
code        String          //
DUP_PARTY|UNMAPPED_GROUP|TB_UNBALANCED|BAD_GSTIN
entity      String
rowNumber   Int?
field       String?
message     String
suggestion  String?
resolvedAt  DateTime?
resolvedByUserId String?

batch MigrationBatch @relation(fields: [batchId], references: [id],
onDelete: Cascade)

@@index([accountId, orgId, batchId, severity])
}

// The proof the migration is correct. Nothing goes live without this
// passing.
model MigrationReconciliation {
  id          String @id @default(cuid())
  accountId   String
  orgId       String
  batchId     String

  checkType   String          //
  TRIAL_BALANCE|PARTY_BALANCE|STOCK_QTY|STOCK_VALUE|TAX_TOTAL
  scopeKey    String?         // partyId / warehouseId / null for totals
  sourceValue Decimal
  targetValue Decimal
  difference   Decimal
  passed       Boolean
  checkedAt    DateTime @default(now())

  batch MigrationBatch @relation(fields: [batchId], references: [id],
onDelete: Cascade)

  @@index([accountId, orgId, batchId, passed])
}

```

Also add to every core entity (`Customer`, `Vendor`, `Item`, `Account`, `Invoice`, `Bill`, ...) so re-runs and post-migration support are possible:

```

    sourceSystem String?          // "TALLY" | "ZOH0" | null when created
in-app
    sourceKey    String?          // the source's own identifier
    migrationBatchId String?

    @@unique([orgId, sourceSystem, sourceKey])

```

3.6 CORE TABLES THE MIGRATION LANDS IN

Migration is only as good as the schema it targets. These are the ledger tables ROADMAP Phase 1 calls for — sketched here because waves 5–8 are meaningless without them.


```

model FiscalYear {
  id          String @id @default(cuid())
  accountId   String
  orgId       String
  name        String          // "2026-27"
  startDate   String
  endDate     String
  status      String @default("OPEN") // OPEN | CLOSED
  lockedThrough String?       // period lock date

  @@unique([orgId, name])
}

model Journal {
  id          String @id @default(cuid())
  accountId   String
  orgId       String
  branchId    String?
  code        String          // SAL | PUR | BNK | CSH | JV | OPN
  name        String
  type        String          // SALE|PURCHASE|BANK|CASH|GENERAL|OPENING
  defaultAccountId String?

  @@unique([orgId, branchId, code])
}

model LedgerAccount {
  id          String @id @default(cuid())
  accountId   String
  orgId       String
  branchId    String?        // null = shared across branches
  code        String
  name        String
  accountType String          // ASSET|LIABILITY|EQUITY|INCOME|EXPENSE
  groupId     String?
  parentId    String?
  isControl   Boolean @default(false) // AR / AP / stock / tax control
  accounts
    controlKind String?       //
    AR|AP|STOCK|CGST_OUT|SGST_OUT|IGST_OUT|...
  currency    String @default("INR")
  isActive    Boolean @default(true)

  sourceSystem String?
  sourceKey    String?

```

```

    @@unique([orgId, branchId, code])
    @@index([accountId, orgId, controlKind])
}

// Control accounts are found by controlKind, never by name string.
// This is the single fix for REVIEW.md gap A-3.

model JournalEntry {
  id          String   @id @default(cuid())
  accountId   String
  orgId       String
  branchId    String
  journalId   String
  fiscalYearId String

  entryNo     String
  date        String
  narration    String?
  status      String   @default("DRAFT") // DRAFT | POSTED | REVERSED
  postedAt    DateTime?
  postedByUserId String?
  reversedById String?

  // What produced this entry
  sourceDocType String? //
  INVOICE|BILL|PAYMENT|RECEIPT|TRANSFER|OPENING
  sourceDocId   String?

  // Tamper-evidence: hash of (prev hash + canonical entry payload)
  prevHash     String?
  hash         String?

  lines        JournalLine[]

  createdByUserId String
  createdAt     DateTime @default(now())
  updatedAt     DateTime @updatedAt

  @@unique([orgId, branchId, entryNo])
  @@index([accountId, orgId, branchId, date])
  @@index([accountId, orgId, sourceDocType, sourceDocId])
}

model JournalLine {
  id          String   @id @default(cuid())

```

```

accountId      String
orgId           String
branchId        String
entryId         String

ledgerAccountId String
partyType       String?           // CUSTOMER | VENDOR
partyId         String?
itemId          String?
warehouseId     String?
costCentreId    String?

debit           Decimal @default(0)
credit          Decimal @default(0)
currency        String  @default("INR")
fxRate          Decimal @default(1)

taxCode         String?           // GST rate/tag for the tax report
hsnSac          String?
description     String?
reconciledWithId String?         // bank reconciliation / payment
allocation

    entry JournalEntry @relation(fields: [entryId], references: [id],
onDelete: Cascade)

    @@index([accountId, orgId, branchId, ledgerAccountId])
    @@index([accountId, orgId, partyType, partyId])
    @@index([entryId])
}
// Invariant enforced in the service layer and by a DB
constraint/trigger:
// SUM(debit) = SUM(credit) per entry, and POSTED entries are immutable.

model PaymentAllocation {
    id          String  @id @default(cuid())
    accountId   String
    orgId       String
    branchId    String
    paymentId   String
    docType     String   // INVOICE | BILL
    docId       String
    amount      Decimal
    createdAt   DateTime @default(now())

    @@index([accountId, orgId, docType, docId])

```

```

    @@index([accountId, orgId, paymentId])
  }

  model StockLayer {
    // valuation, FIFO/AVCO
    id          String @id @default(cuid())
    accountId   String
    orgId       String
    branchId    String
    warehouseId String
    itemId      String

    date        String
    qtyIn       Decimal @default(0)
    qtyRemaining Decimal @default(0)
    unitCost    Decimal
    sourceDocType String?
    sourceDocId String?

    @@index([accountId, orgId, branchId, warehouseId, itemId, date])
  }

```

3.7 MIGRATION AS A PRODUCT, NOT A SERVICE

- **Self-serve first:** upload → map → dry run → report, with no human involved, for Excel and Zoho sources.
- **Assisted for Tally:** a small desktop helper (or a documented XML export recipe) that pulls masters, trial balance, open items, and stock summary in one click, and uploads them.
- **The reconciliation PDF is the sales artifact.** “Your trial balance matched to ₹0.00 across 1,847 ledgers” closes deals that feature lists do not.
- **Free migration** as the standing offer. It costs you engineering once and removes the single largest objection every time.
- **Keep the source files** for the retention period. When a customer disputes a number six months later, you need the original export.

4. What to do with this

Sequenced against the existing roadmap:

Phase	Add from this document
Phase 1 (server as system of record)	Core tables in 3.6 — this <i>is</i> the ledger work, now with migration and control-account requirements baked in. Add <code>sourceSystem</code> / <code>sourceKey</code> to every entity from day one; retrofitting it later is painful
Phase 1.5 (new)	Migration engine: Excel/CSV ingestion, mapping, dry run, reconciliation, tables in 3.5. Ship before the first paying customer, not after
Phase 2 (trustworthy books)	Numbering series (C2), regional settings (C15), approval workflows (C6)
Phase 3 (hardening)	Custom fields (C1), print templates (C3), saved views (C7), message templates (C9), record rules (C5)
Phase 4 (compliance)	Tally XML adapter, Zoho connector, delta runs
Phase 5 (scale)	Automations (C10), dashboards (C8), API keys/webhooks (C14), voucher types (C16), CA multi-client console

The competitive answer and the migration answer are the same answer: **be the product that is easy to arrive at and impossible to outgrow at the branch level.** Everything in Section 1.3 and Section 3 serves that one sentence.